Module 8 — Model Lifecycle, Versioning & Safe Updates in FastAPI

Learning Objectives

By the end of this module, you will be able to:

- Manage the **Lifecycle** of an AI model from startup loading to graceful shutdown.
- Implement **Versioning Strategies** using FastAPI's `APIRouter` to support multiple model versions simultaneously.
- Utilize **Dependency Injection** to safely swap or "inject" specific model versions into your inference endpoints.
- Design **Safe Update** workflows that prevent service downtime and memory crashes during model replacement.
- Maintain **API Trust** by providing consistent, backward-compatible prediction results for existing users.

The Model Lifecycle in Production

In an AI Engineering context, a model's life does not end once it is trained. In a production **FastAPI** environment, the lifecycle is a continuous loop.

1. **Training & Validation:** The model is built and tested in a research environment (like a notebook).
2. **Deployment (Startup):** The model is packaged and loaded into server memory using **Startup Events**.
3. **Monitoring:** The API logs performance and prediction confidence (Module 7).
4. **Replacement/Update:** A new version is rolled out. Blindly replacing the old model file is dangerous because ML models consume significant **Memory**; loading a new 2GB model while the old 2GB model is still active could exceed container limits and crash the service.

Model Versioning Strategies

AI Engineers avoid "hot-swapping" models blindly. Instead, they use structured versioning to ensure that if a new model fails, the system can instantly revert to the old one.

1. Versioned Prediction Endpoints

Using the **APIRouter**, you can create separate "Path Prefixes" for different model versions.

- **Why this exists:** To allow users to migrate to a new model at their own pace.
- **Where it is used:** When a new model version has a different input schema or produces different results (e.g., changing from a 3-class classifier to a 5-class classifier).
- **What breaks:** If you replace the model on the same path, existing client apps might receive unexpected results, breaking their internal logic.

2. Dependency Injection for Models

FastAPI's **Dependency Injection** system allows your endpoints to "declare" which model version they require to work.

- **Why this exists:** To share model loading logic across multiple routes and manage model versions centrally.
- **Where it is used:** Injecting the "production-stable" model into the main `/predict` path while allowing internal testers to access a "beta" model via a different path using the same logic.

Code Examples & Line-by-Line Explanations

Example 1: Loading Multiple Versions on Startup

This code uses **Startup Events** to ensure all necessary model versions are ready before the API accepts traffic.

```
from fastapi import FastAPI
import logging

# Global registry for models
model_registry = {}

app = FastAPI()

@app.on_event("startup") # (1)
def load_all_models():
    # Simulate loading two different versions of weights
    model_registry["v1"] = {"name": "RandomForest", "accuracy": 0.85}
    model_registry["v2"] = {"name": "XGBoost", "accuracy": 0.92}
    logging.info("All model versions loaded into memory.") # (2)

@app.get("/models")
def list_models():
    return model_registry
```

Line-by-Line Explanation:

1. **`@app.on_event("startup")`**: This ensures models are loaded **once** when the server starts.
2. **`logging.info(...)`**: Provides observability so the AI Engineer knows the models are ready for inference (Module 7).

Example 2: Versioned Routes with APIRouter

This matches the "Bigger Applications" pattern to isolate model versions.

```
from fastapi import FastAPI, APIRouter # (1)

app = FastAPI()

# --- v1 Logic ---
v1_router = APIRouter(prefix="/v1") # (2)
@v1_router.post("/predict")
def predict_v1():
    return {"model": "v1_stable", "prediction": 100}

# --- v2 Logic ---
v2_router = APIRouter(prefix="/v2") # (3)
@v2_router.post("/predict")
def predict_v2():
    return {"model": "v2_improved", "prediction": 105.2}

# Registering the routers
app.include_router(v1_router) # (4)
app.include_router(v2_router)
```

Line-by-Line Explanation:

1. **`APIRouter`**: A tool to group related paths together.
2. **`prefix="/v1"`**: Automatically adds `/v1` to all routes inside this router.
3. **`v2_router`**: Hosts the improved model without deleting the old logic.
4. **`app.include_router(...)`**: Plugs the versioned routes into the main application.

Example 3: Switching Models Safely via Dependency Injection

This uses the **Depends** system to "inject" a specific model version into a function.

```
from typing import Annotated
from fastapi import Depends, FastAPI

app = FastAPI()

# The "Dependable" function
def get_latest_model(): # (1)
    # This could be logic that checks a config file or environment variable
    return "XGBoost_v2"

@app.post("/predict")
def predict(model: Annotated[str, Depends(get_latest_model)]): # (2)
    return {"active_model": model, "result": "success"}
```

Line-by-Line Explanation:

1. **`get_latest_model()`**: A dependency function that decides which model is currently "Active".

2. **`Depends(get_latest_model)`**: FastAPI calls this function for every request and passes the result into the `model` parameter. This allows the AI Engineer to change the active model in **one place** without touching the endpoint code.

Real-World AI Scenarios

- **Rolling out Improved Accuracy Model:** You deploy a new model at `/v2/predict`. You monitor its logs (Module 7) while users are still safely using `/v1/predict`. Once you are confident, you update your **Dependency Injection** logic to point the main `/predict` path to the `v2` weights.
- **Emergency Rollback:** If `/v2` starts giving "garbage" results or high latency, you don't need to redeploy the whole server. You simply change the `get_latest_model` dependency to return the `v1` string again and restart the service.
- **Trust and Contracts:** By using **Response Models**, you ensure that even if the internal math of the model changes from `v1` to `v2`, the JSON output structure remains identical, so the user's dashboard doesn't break.

Common Mistakes AI Engineers Make

- **Hot-Swapping Blindly:** Overwriting a `model.pkl` file on a running server. This can lead to "File Not Found" errors or corrupt memory states during a request.
- **Ignoring Memory Limits:** Loading multiple versions of a heavy model (e.g., two 4GB models) on a server with only 6GB of RAM.
- **Hardcoding Versions:** Putting the version number inside the inference function logic instead of using **Routers** or **Dependencies**.
- **Forgetting Shutdown Events:** Failing to close large file handles or GPU sessions when the server stops.

Assignment: Add Versioned Model Endpoints

1. Create a FastAPI app with a global dictionary `loaded_models`.
2. On **Startup**, load a dummy dictionary for `"v1"` and `"v2"`.
3. Create two `APIRouter` instances: one with prefix `/stable` and one with prefix `/beta`.
4. Implement a `/predict` POST endpoint in each router.
5. The `/stable/predict` endpoint must use the `v1` model data.
6. The `/beta/predict` endpoint must use the `v2` model data.
7. Verify in the **Swagger UI (/docs)** that both versions are visible and usable.

Mini Project: Multi-Version Inference Service

Goal: Build a "Smooth Rollout" API.

- Define a Pydantic `InferenceResult` schema (Module 3).
- Create a dependency `get_active_version` that reads a string from a local file (`config.txt`).
- Create a POST `/inference` endpoint that uses the `get_active_version` dependency to decide which "dummy weights" to use.
- Update the `config.txt` file and observe how the API response changes without you modifying the `main.py` code.

Deployment Safety Checklist: Is my model update-safe?

- [ ] Have I confirmed the **Memory usage** of loading both the new and old model simultaneously?
- [ ] Is the new model being served on a **new versioned path** (e.g., `/v2`) or via **Dependency Injection**?
- [ ] Does the new version use a **Response Model** to ensure backward compatibility for the output JSON?
- [ ] Are models being loaded in a **Startup Event** to avoid per-request latency?
- [ ] Is there a **Rollback plan** (e.g., a simple config change) if the new version fails in production?
- [ ] Are all model versions integrated into the **OpenAPI documentation** for transparency?